25

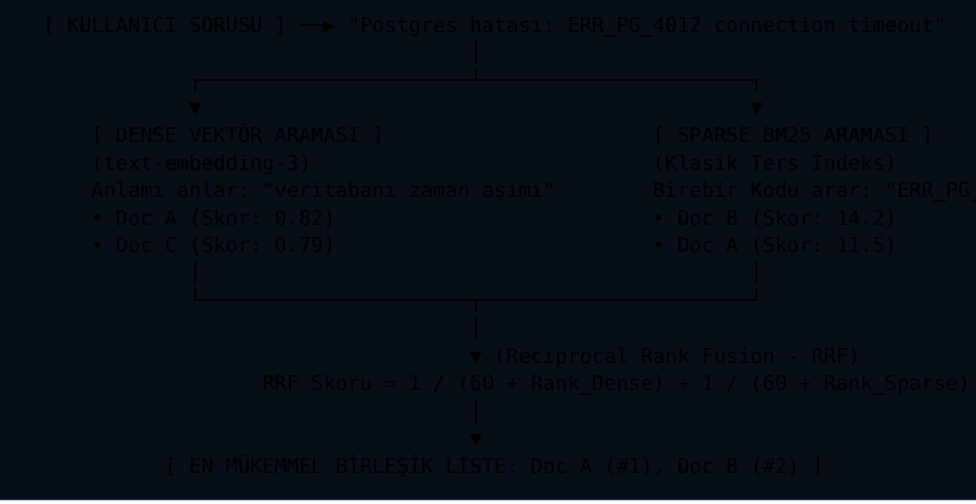
BÖLÜM: İLERI DÜZEY RAG (ADVANCED RAG) & BILGI GETIRME OPTIMIZASYONU | MODULE: ADVANCED RAG & INFORMATION RETRIEVAL OPTIMIZATION

Hibrit Arama (Hybrid Search): Vektör (Dense) + BM25 (Sparse) ile Reciprocal Rank Fusion

Hybrid Search: Combining Dense Vector Embeddings with Sparse BM25 via RRF

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir dedektifin tek bir yardımcısı olursa işler aksayabilir. Biri felsefeden ve hislerden anlayan bir psikolog olsun (Vektör / Dense Search: 'Kullanıcı veritabanı kilitlenmesinden bahsediyor'). Diğeri ise sadece plaka ve kimlik numarası ezberleyen bir parmak izi uzmanı olsun (BM25 / Sparse Search: 'Aranan hata kodu tam olarak ERR_PG_4012!'). İkisi bir araya geldiğinde (Hibrit Arama), psikolog konunun özünü anlarken parmak izi uzmanı tam nokta atışı hata kodunu yakalar! Tek başına vektör araması hata kodlarını ve seri numaralarını kaçırır; BM25 ile birleşince yenilmez olur!

🇬🇧 ENGLISH (ELI5)

Imagine a murder investigation with two specialized detectives: Detective Vector evaluates psychological motive and semantic themes ('The killer felt betrayed'), while Detective BM25 searches for exact fingerprint serial numbers ('Weapon registered to ID 9481'). Pure vector search excels at high-level concepts but stumbles on exact alphanumeric product codes, API tokens, and error strings. Hybrid Search fuses Dense Vector embeddings with Sparse BM25 keyword matching via Reciprocal Rank Fusion (RRF), capturing both semantic nuance and exact lexical matches!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Hibrit Arama ve RRF Matematiği: 1) Dense Vector (Yoğun Vektör): 1536 float sayısıdır; anlam benzerliğini yakalar ama 'iPhone 15 Pro Max 256GB' ile 'iPhone 14 Pro Max 256GB' arasındaki minik rakam farkını kaçırabilir. 2) Sparse Vector (BM25 - Seyrek Vektör): TF-IDF'in gelişmiş halidir; nadir kelimelere yüksek ağırlık verir, exact match'i yakalar. 3) Reciprocal Rank Fusion (RRF): Farklı ölçeklerdeki skorları (Cosine 0-1 ile BM25 0-50) doğrudan toplayamazsınız! RRF skorları değil, sıralama sıralarını (Rank) birleştirir: RRF(d) = Σ [1 / (k + rank_m(d))]. Genellikle k = 60 sabiti kullanılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Hybrid Search Topology & Reciprocal Rank Fusion (RRF): 1) Dense Vectors: Project text into continuous semantic manifolds. Prone to precision collapse on exact alphanumeric SKU tokens, legal clauses, and stack traces. 2) Sparse BM25 Vectors: Exact lexical term frequency-inverse document frequency weighting. 3) Normalization Impedance: Raw cosine scores (0.0 to 1.0) cannot be naively added to unbounded BM25 scores (0 to 45). RRF solves this by operating strictly on ordinal rank positions: RRF(d) = Σ [1 / (k + rank_m(d))], where k=60 dampens the influence of top outliers across heterogeneous result lists.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25: Reciprocal Rank Fusion (RRF) Algoritması ile Liste Birleştirme
import java.util.*;

public class ReciprocalRankFusion {
    private static final int K = 60; // Standart RRF yumuşatma sabiti

    public record SearchResult(String docId, double score) {}

    public static List<SearchResult> mergeRrf(List<String> denseRankedIds, List<String>
sparseRankedIds, int topN) {
        Map<String, Double> rrfScores = new HashMap<>();

        // 1. Dense sıralamasını işle
        for (int rank = 0; rank < denseRankedIds.size(); rank++) {
            String docId = denseRankedIds.get(rank);
            rrfScores.put(docId, rrfScores.getOrDefault(docId, 0.0) + (1.0 / (K + rank + 1)));
        }

        // 2. Sparse (BM25) sıralamasını işle
        for (int rank = 0; rank < sparseRankedIds.size(); rank++) {
            String docId = sparseRankedIds.get(rank);
            rrfScores.put(docId, rrfScores.getOrDefault(docId, 0.0) + (1.0 / (K + rank + 1)));
        }

        // 3. Skorlara göre azalan sırada sırala ve ilk TopN sonucu al
        return rrfScores.entrySet().stream()
            .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
            .limit(topN)
            .map(e -> new SearchResult(e.getKey(), e.getValue()))
            .toList();
    }
}

void main() {
    List<String> vectorResults = List.of("DocA", "DocB", "DocC");
    List<String> bm25Results   = List.of("DocB", "DocA", "DocD");

    var combined = ReciprocalRankFusion.mergeRrf(vectorResults, bm25Results, 3);
    for (var res : combined) {
        IO.println("Doküman: " + res.docId() + " | RRF Skoru: " + String.format("%.5f", res.score()));
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Sorusu: 'Neden Cosine benzerlik skoru ile BM25 skorunu doğrudan toplayıp ortalamasını almıyoruz?' Cevap: 'Çünkü iki algoritmanın skor dağılımları ve varyansları tamamen farklıdır (Normalizasyon Uyuşmazlığı). Doğrudan toplamak bir metriğin diğerini ezmesine yol açar; bu yüzden rütbe tabanlı RRF zorunludur.'

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Interview trap: 'Why not simply average the Cosine and BM25 scores together?' Decisive answer: 'Cosine and BM25 follow incompatible statistical probability distributions. Raw score summation leads to one distribution dominating the other. Rank-based RRF is scale-invariant and immune to distribution skew.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Hibrit arama (Dense + Sparse), kurumsal RAG'de bilgi getirme doğruluğunu %30 artırır.
- BM25 exact match (hata kodları, seri no) için; Vektör ise kavramsal anlam için şarttır.
- Reciprocal Rank Fusion (RRF), farklı arama motorlarının sonuçlarını adilce birleştirir.

🎯 Key Takeaways for the Interview

- Hybrid search delivers a 20-30% recall boost over pure dense vector retrieval in enterprise domains.
- BM25 guarantees exact keyword hits; dense vectors provide semantic generalization.
- RRF provides a mathematically robust, scale-invariant rank-fusion algorithm.